Project Report

on

Bookstore Management System
Advance Database Management System (26CAP-602)

Submitted by

Rishabh Shakya

26MCI10167

Under the guidance of

Mr.Chanchal Sharma

in partial fulfilment for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING



Chandigarh University

November 2026

Certificate

This is to certify that **Rishabh Shakya**, a student of **Master of Computer Applications (MCA) – Artificial Intelligence & Machine Learning**, has successfully completed the **Minor Project** titled **"Bookstore Management System"** under the esteemed guidance of **Mr. Chanchal Sharma, Assistant Professor, University Institute of Computing (UIC), Chandigarh University**.

This project was undertaken as a part of the academic curriculum and is submitted in **partial fulfilment of the requirements** for the MCA program. The work presented in this project is a result of **independent research, diligent effort, and dedication**, demonstrating the student's ability to apply theoretical knowledge to practical problem-solving.

The project, "Bookstore Management System" is a web-based collaborative platform that integrates real-time communication, Pomodoro-based productivity tracking, and resource sharing. It effectively showcases concepts of Advanced Database Management System, real-time communication, and database-driven architecture.

I hereby confirm that this project is an **original work** carried out by the student and has **not been submitted elsewhere** for the award of any other degree, diploma, or certification.

Project Guide:
Mr. Chanchal Sharma
Assistant Professor
University Institute of Computing
Chandigarh University

Acknowledgement

I would like to express my sincere gratitude to **Chandigarh University** and the **University Institute of Computing (UIC)** for providing me with the opportunity to undertake this project, **"Bookstore Management System"**

I extend my heartfelt appreciation to my esteemed mentor, **Mr. Chanchal Sharma, Assistant Professor**, for his invaluable guidance, continuous support, and insightful feedback throughout the project. His expertise in **Advanced Database Management System** played a crucial role in the successful completion of this project.

I am also grateful to my friends and peers for their encouragement and discussions, which helped refine my approach. Lastly, I thank my family for their unwavering support and motivation during this research.

This project has been an incredible learning experience, and I hope it serves as a foundation for further exploration in **Advanced Database Management System.**

Rishabh Shakya

MCA – Artificial Intelligence & Machine Learning

Chandigarh University

Contents

Section	Page Number
1. Introduction	1
2. Objective	2
3. Tools and Libraries Used	3
4. Implementation Steps	4
5. Screenshots / Results	5 – 14
6. Conclusion	15
7. References	16

1. Introduction

The **Online Bookstore Management System** is a secure, efficient, and robust database project designed to store, manage, and process book catalog, customer, and order records in an **Oracle PL/SQL** environment. It aims to modernize retail book sales by combining the principles of **Database Management Systems (DBMS)** with procedural programming. Using a **bottom-up design approach**, the system builds a strong relational foundation by defining core entities such as **Users, Books, Shopping Cart, Orders, and Order Items**, ensuring consistency, accuracy, and operational flexibility across the entire bookstore workflow.

The project strictly implements **normalization up to the Third Normal Form (3NF)** to eliminate data redundancy, prevent insertion or update anomalies, and enforce referential integrity between tables. To optimize search performance, **indexing** is applied to primary attributes like ISBN, Title, and Category, significantly reducing query response time. Furthermore, the system demonstrates the use of **relational database operations**—including selection, projection, joins, and aggregate functions—to retrieve catalog data and compute real-time sales summaries.

At the core of the system is the **PL/SQL procedural engine**, which automates business logic directly inside the database. **Stored Procedures and Stored Functions** are implemented for user registration, stock verification, and cart calculations. The system utilizes **explicit cursors** to iterate through multi-item shopping carts line by line. Additionally, **event-driven database triggers** enforce critical business rules: preventing negative stock levels and automatically restoring book inventory when an order is cancelled.

A dedicated **transaction manager** ensures that all checkout operations adhere strictly to **ACID (Atomicity, Consistency, Isolation, Durability)** properties. If an error occurs during checkout, the transaction is safely rolled back to preserve data integrity. Overall, the **Online Bookstore Management System** serves as a reliable and practical solution, seamlessly combining theoretical DBMS concepts with real-world **Oracle PL/SQL** application.

.

2. Objective

1. To design a robust and scalable online bookstore database system:

Develop a reliable relational database that efficiently manages book catalog, customer accounts, shopping cart items, and billing records while ensuring data privacy and operational reliability.

2. To achieve database normalization:

Apply normalization principles (**1NF, 2NF, and 3NF**) to eliminate data redundancy, prevent modification anomalies, and optimize overall database storage and query performance.

3. To implement PL/SQL stored procedures and functions:

Encapsulate critical business operations directly inside the database by developing **stored procedures** for user registration and order processing, and **stored functions** for real-time stock validation and cart total calculations.

4. To automate inventory management using database triggers:

Implement **event-driven triggers** to enforce business rules automatically, preventing negative book stock and restocking inventory seamlessly whenever an order status is marked as cancelled.

5. To demonstrate cursor-based multi-row processing :

Utilize **explicit cursors** to iterate line-by-line through multi-item shopping carts during checkout and generate formatted catalog reports.

6. To ensure transaction management and ACID compliance:

Guarantee database reliability through atomic transactions using `COMMIT` and `ROLLBACK` commands during concurrent customer checkout operations.

7. To implement role-based access control and security:

Provide role-based segregation between **Customers** and **Administrators** while securing user credentials through hashed password storage.

3. Tools and Technologies Used

The **Online Bookstore Management System** is developed using a robust combination of database management technologies and programming tools to ensure security, transactional accuracy, and efficient data processing. The primary backend technology used is the **Oracle Database (11g/19c/21c/LiveSQL)**, which serves as the core relational engine for storing book catalog, user account, shopping cart, and billing records. The **SQL** language is utilized for schema definition, constraints, normalization, and executing relational queries involving joins, projections, and aggregations.

At the core of the system is **Oracle PL/SQL (Procedural Language/SQL)**, which executes procedural business logic directly within the database server. PL/SQL is used to develop **stored procedures, stored functions, explicit cursors, and event-driven triggers**. For database design and architectural modeling, tools like **Draw.io** and **Oracle SQL Developer Data Modeler** are used to create the **Entity-Relationship (ER) Diagram** and map relational schemas.

For frontend and interface demonstration, the project uses **HTML5, CSS3, and modern JavaScript**, connected via a lightweight **Python backend server**. This architecture enables real-time visual feedback for database operations, allowing users to browse books, manage carts, and observe live PL/SQL execution logs.

In terms of advanced database management, **database triggers** are implemented for automated stock replenishment upon order cancellation and negative inventory prevention. Concurrency control and reliability are maintained through an **ACID-compliant transaction manager** utilizing `COMMIT` and `ROLLBACK` commands. Overall, the selected tools and technologies make the system a practical, secure, and complete demonstration of **Oracle PL/SQL** capabilities.

4. Implementation Steps

- **Requirement Analysis:**
  Identify the core entities and operational requirements of the online bookstore system. The primary entities include **Users (Customer and Admin), Books, Categories, Shopping Cart, Orders, and Order Items**. Define essential attributes, data types, validation constraints, and workflow operations.

- **Conceptual Design (ER Diagram):**
  Construct an **Entity-Relationship (ER) Diagram** to visually represent entities, attributes, and cardinality relationships, such as *User adds Books to Cart*, *User places Order*, and *Order contains Order Items*. This provides a clear architectural blueprint for the database.

- **Logical Design and Normalization:**
  Map the ER diagram into relational tables and apply **Normalization (up to 3NF)** to remove data redundancy, prevent update anomalies, and maintain consistency. Establish **Primary Keys, Foreign Keys**, and referential constraints across all tables.

- **Schema Implemantation (DDL & DML):**
  Execute SQL commands in **Oracle Database** to create tables and auto-incrementing **Sequences** (`USER_SEQ`, `BOOK_SEQ`, `ORDER_SEQ`). Define integrity constraints (`CHECK`, `UNIQUE`, `NOT NULL`) and populate the catalog with sample books and user records using DML commands.

- **PL/SQL Development (Procedures, Functions & Cursor):**
  Implement business logic directly within the database using **Stored Procedures** for registration and checkout, **Stored Functions** for real-time stock checks and cart totals, and **Explicit Cursors** to iterate line-by-line through multi-item carts during order placement.

- **Trigger-Based Automation:**
  Create **event-driven database triggers** to enforce business rules automatically: `trg_check_stock` prevents negative inventory counts, while `trg_restore_stock_on_cancel` automatically replenishes book stock whenever an order is marked as cancelled.

- **Transaction Management & ACID Compliance:**
  Incorporate atomic transaction control using `COMMIT`, `ROLLBACK`, and `SAVEPOINT` statements. This guarantees **ACID properties** during checkout, ensuring that cart clearing and inventory deduction happen completely or not at all in case of error.

- **Testing and Validation:**
  Validate all procedures, functions, and triggers against comprehensive test scenarios. Verify duplicate email rejection, out-of-stock validation, order placement, automatic trigger execution on cancellation, and exception handling.

5. Screenshots/Result

bookstore_student.sql

```sql
-- ================================================================================
-- ONLINE BOOKSTORE MANAGEMENT SYSTEM (PL/SQL)
-- STUDENT LEVEL MINI PROJECT (Easy to Understand & Explain in Viva)
-- ================================================================================
-- This single script contains:
-- 1. Tables (Users, Books, Cart, Orders, Order_Items)
-- 2. Sequences for Auto IDs
-- 3. Stored Procedures (Registration, Add to Cart, Place Order)
-- 4. Stored Functions (Calculate Total, Stock Check)
-- 5. Database Triggers (Auto stock deduction, Restock on cancel)
-- 6. Cursor implementation (Displaying books)
-- 7. Sample Data & Test Execution
-- ================================================================================

SET SERVEROUTPUT ON SIZE UNLIMITED;

-- --------------------------------------------------------------------------------
-- STEP 1: CLEANUP OLD TABLES IF THEY ALREADY EXIST
-- --------------------------------------------------------------------------------
BEGIN
    FOR t IN (SELECT table_name FROM user_tables WHERE table_name IN
              ('ORDER_ITEMS', 'ORDERS', 'CART', 'BOOKS', 'USERS')) LOOP
        EXECUTE IMMEDIATE 'DROP TABLE ' || t.table_name || ' CASCADE CONSTRAINTS';
    END LOOP;

    FOR s IN (SELECT sequence_name FROM user_sequences WHERE sequence_name IN
              ('USER_SEQ', 'BOOK_SEQ', 'ORDER_SEQ', 'CART_SEQ', 'ITEM_SEQ')) LOOP
        EXECUTE IMMEDIATE 'DROP SEQUENCE ' || s.sequence_name;
    END LOOP;
EXCEPTION
    WHEN OTHERS THEN
        NULL;
END;
/

-- --------------------------------------------------------------------------------
-- STEP 2: SEQUENCES (Auto-generating IDs)
-- --------------------------------------------------------------------------------
CREATE SEQUENCE USER_SEQ START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE BOOK_SEQ START WITH 101 INCREMENT BY 1;
CREATE SEQUENCE CART_SEQ START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE ORDER_SEQ START WITH 5001 INCREMENT BY 1;
CREATE SEQUENCE ITEM_SEQ START WITH 1 INCREMENT BY 1;

-- --------------------------------------------------------------------------------
-- STEP 3: CREATING TABLES
-- --------------------------------------------------------------------------------

-- Table 1: USERS (Customer aur Admin ki details)
CREATE TABLE USERS (
    user_id       NUMBER PRIMARY KEY,
    name          VARCHAR2(100) NOT NULL,
    email         VARCHAR2(100) UNIQUE NOT NULL,
    password      VARCHAR2(50) NOT NULL,
    phone         VARCHAR2(15),
    role          VARCHAR2(10) DEFAULT 'CUSTOMER',  -- 'CUSTOMER' ya 'ADMIN'
    created_date  DATE DEFAULT SYSDATE
);

-- Table 2: BOOKS (Book catalog and stock)
CREATE TABLE BOOKS (
    book_id       NUMBER PRIMARY KEY,
    title         VARCHAR2(150) NOT NULL,
    author        VARCHAR2(100) NOT NULL,
    category      VARCHAR2(50),
    price         NUMBER(8,2) NOT NULL,
    stock         NUMBER(5) DEFAULT 0,
    CONSTRAINT chk_price CHECK (price > 0),
    CONSTRAINT chk_stock CHECK (stock >= 0)
);

-- Table 3: CART (Shopping cart items)
CREATE TABLE CART (
    cart_id       NUMBER PRIMARY KEY,
    user_id       NUMBER REFERENCES USERS(user_id) ON DELETE CASCADE,
    book_id       NUMBER REFERENCES BOOKS(book_id) ON DELETE CASCADE,
    quantity      NUMBER DEFAULT 1,
    CONSTRAINT chk_qty CHECK (quantity > 0)
);

-- Table 4: ORDERS (Order summary)
CREATE TABLE ORDERS (
    order_id      NUMBER PRIMARY KEY,
    user_id       NUMBER REFERENCES USERS(user_id),
    order_date    DATE DEFAULT SYSDATE,
    total_amount  NUMBER(10,2) NOT NULL,
    order_status  VARCHAR2(20) DEFAULT 'PLACED'  -- 'PLACED', 'CANCELLED'
);

-- Table 5: ORDER_ITEMS (Books inside each order)
CREATE TABLE ORDER_ITEMS (
    item_id       NUMBER PRIMARY KEY,
    order_id      NUMBER REFERENCES ORDERS(order_id) ON DELETE CASCADE,
    book_id       NUMBER REFERENCES BOOKS(book_id),
    quantity      NUMBER NOT NULL,
    unit_price    NUMBER(8,2) NOT NULL
);

PROMPT Tables and Sequences created successfully!
```

```
-- ------------------------------------------------------------------------------
-- STEP 4: PL/SQL FUNCTIONS
-- ------------------------------------------------------------------------------

-- Function 1: Check if book is in stock
-- Concept: Function hamesha ek value RETURN karta hai.
CREATE OR REPLACE FUNCTION check_book_stock(
    p_book_id IN NUMBER,
    p_qty     IN NUMBER
) RETURN BOOLEAN IS
    v_available NUMBER := 0;
BEGIN
    SELECT stock INTO v_available
    FROM BOOKS
    WHERE book_id = p_book_id;

    IF v_available >= p_qty THEN
        RETURN TRUE;
    ELSE
        RETURN FALSE;
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN FALSE;
END;
/

-- Function 2: Calculate Cart Total for a User
CREATE OR REPLACE FUNCTION get_cart_total(
    p_user_id IN NUMBER
) RETURN NUMBER IS
    v_total NUMBER := 0;
BEGIN
    SELECT NVL(SUM(c.quantity * b.price), 0)
    INTO v_total
    FROM CART c
    JOIN BOOKS b ON c.book_id = b.book_id
    WHERE c.user_id = p_user_id;

    RETURN v_total;
END;
/

PROMPT Functions created successfully!

-- ------------------------------------------------------------------------------
-- STEP 5: PL/SQL PROCEDURES
-- ------------------------------------------------------------------------------

-- Procedure 1: Register New User
-- Concept: Procedure koi action perform karta hai (INSERT/UPDATE).
CREATE OR REPLACE PROCEDURE register_user(
    p_name     IN VARCHAR2,
    p_email    IN VARCHAR2,
    p_password IN VARCHAR2,
    p_phone    IN VARCHAR2,
    p_role     IN VARCHAR2 DEFAULT 'CUSTOMER'
) IS
BEGIN
    INSERT INTO USERS (user_id, name, email, password, phone, role)
    VALUES (USER_SEQ.NEXTVAL, p_name, LOWER(p_email), p_password, p_phone, UPPER(p_role));

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Success: User ' || p_name || ' registered successfully!');
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('Error: Email ' || p_email || ' already registered!');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error in registration: ' || SQLERRM);
END;
/

-- Procedure 2: Add Book to Cart with Stock Verification
CREATE OR REPLACE PROCEDURE add_to_cart(
    p_user_id IN NUMBER,
    p_book_id IN NUMBER,
    p_quantity IN NUMBER DEFAULT 1
) IS
    v_exists NUMBER := 0;
BEGIN
    -- Check if stock is available using our function
    IF NOT check_book_stock(p_book_id, p_quantity) THEN
        DBMS_OUTPUT.PUT_LINE('Error: Requested quantity not available in stock.');
        RETURN;
    END IF;

    -- Check if item is already in cart
    SELECT COUNT(*) INTO v_exists
    FROM CART
    WHERE user_id = p_user_id AND book_id = p_book_id;

    IF v_exists > 0 THEN
        UPDATE CART
        SET quantity = quantity + p_quantity
        WHERE user_id = p_user_id AND book_id = p_book_id;
        DBMS_OUTPUT.PUT_LINE('Updated cart quantity.');
    ELSE
        INSERT INTO CART (cart_id, user_id, book_id, quantity)
        VALUES (CART_SEQ.NEXTVAL, p_user_id, p_book_id, p_quantity);
        DBMS_OUTPUT.PUT_LINE('Book added to cart successfully.');
    END IF;
```

```
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error adding to cart: ' || SQLERRM);
END;
/

-- Procedure 3: Place Order from Cart (ATOMIC TRANSACTION)
-- Concept: Cursor use karta hai cart ke saare items iterate karne ke liye.
CREATE OR REPLACE PROCEDURE place_order(
    p_user_id IN NUMBER
) IS
    v_order_id NUMBER;
    v_total    NUMBER;
    v_cart_cnt NUMBER;

    -- EXPLICIT CURSOR: cart ke saare books padhne ke liye
    CURSOR c_cart IS
        SELECT c.book_id, c.quantity, b.price, b.title, b.stock
        FROM CART c
        JOIN BOOKS b ON c.book_id = b.book_id
        WHERE c.user_id = p_user_id;

BEGIN
    -- 1. Check if cart is empty
    SELECT COUNT(*) INTO v_cart_cnt FROM CART WHERE user_id = p_user_id;
    IF v_cart_cnt = 0 THEN
        DBMS_OUTPUT.PUT_LINE('Error: Cart is empty. Cannot place order.');
        RETURN;
    END IF;

    -- 2. Calculate Total
    v_total := get_cart_total(p_user_id);

    -- 3. Verify stock for each item using cursor
    FOR item IN c_cart LOOP
        IF item.stock < item.quantity THEN
            DBMS_OUTPUT.PUT_LINE('Order Failed: Out of stock for book "' || item.title || '"');
            RETURN;
        END IF;
    END LOOP;

    -- 4. Create Order Header
    v_order_id := ORDER_SEQ.NEXTVAL;
    INSERT INTO ORDERS (order_id, user_id, order_date, total_amount, order_status)
    VALUES (v_order_id, p_user_id, SYSDATE, v_total, 'PLACED');

    -- 5. Copy Cart Items to ORDER_ITEMS and deduct stock
    FOR item IN c_cart LOOP
        INSERT INTO ORDER_ITEMS (item_id, order_id, book_id, quantity, unit_price)
        VALUES (ITEM_SEQ.NEXTVAL, v_order_id, item.book_id, item.quantity, item.price);

        -- Deduct inventory
        UPDATE BOOKS
        SET stock = stock - item.quantity
        WHERE book_id = item.book_id;
    END LOOP;

    -- 6. Empty the shopping cart
    DELETE FROM CART WHERE user_id = p_user_id;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('SUCCESS: Order #' || v_order_id || ' placed! Total Amount: Rs.' || v_total);

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Order failed: ' || SQLERRM);
END;
/

-- Procedure 4: Display All Books Using Cursor (Viva examiner Loves this!)
CREATE OR REPLACE PROCEDURE display_all_books IS
    CURSOR book_cur IS
        SELECT book_id, title, author, price, stock
        FROM BOOKS
        ORDER BY book_id;

    v_rec book_cur%ROWTYPE;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--------------------------------------------------------------------------');
    DBMS_OUTPUT.PUT_LINE(RPAD('ID', 6) || RPAD('TITLE', 30) || RPAD('AUTHOR', 20) || RPAD('PRICE', 10) || 'STOCK');
    DBMS_OUTPUT.PUT_LINE('--------------------------------------------------------------------------');

    OPEN book_cur;
    LOOP
        FETCH book_cur INTO v_rec;
        EXIT WHEN book_cur%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            RPAD(v_rec.book_id, 6) ||
            RPAD(SUBSTR(v_rec.title, 1, 28), 30) ||
            RPAD(SUBSTR(v_rec.author, 1, 18), 20) ||
            RPAD('Rs.' || v_rec.price, 10) ||
            v_rec.stock
        );
    END LOOP;
    CLOSE book_cur;
    DBMS_OUTPUT.PUT_LINE('--------------------------------------------------------------------------');
```

```sql
END;
/

PROMPT Procedures created successfully!

-- ------------------------------------------------------------------------
-- STEP 6: PL/SQL TRIGGERS
-- ------------------------------------------------------------------------

-- Trigger 1: Negative stock rokna (BEFORE UPDATE)
CREATE OR REPLACE TRIGGER trg_check_stock
BEFORE INSERT OR UPDATE OF stock ON BOOKS
FOR EACH ROW
BEGIN
    IF :NEW.stock < 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Stock cannot be negative for book: ' || :NEW.title);
    END IF;
END;
/

-- Trigger 2: Agar Order CANCEL ho, toh stock wapas restore karna (AFTER UPDATE)
CREATE OR REPLACE TRIGGER trg_restore_stock_on_cancel
AFTER UPDATE OF order_status ON ORDERS
FOR EACH ROW
BEGIN
    IF :NEW.order_status = 'CANCELLED' AND :OLD.order_status != 'CANCELLED' THEN
        FOR item IN (SELECT book_id, quantity FROM ORDER_ITEMS WHERE order_id = :NEW.order_id) LOOP
            UPDATE BOOKS
            SET stock = stock + item.quantity
            WHERE book_id = item.book_id;
        END LOOP;
        DBMS_OUTPUT.PUT_LINE('Trigger: Inventory automatically restored for cancelled order #' || :NEW.order_id);
    END IF;
END;
/

PROMPT Triggers created successfully!

-- ------------------------------------------------------------------------
-- STEP 7: SAMPLE DATA SEEDING
-- ------------------------------------------------------------------------

-- Insert Books
INSERT INTO BOOKS VALUES (BOOK_SEQ.NEXTVAL, 'Oracle PL/SQL Programming', 'Steven Feuerstein', 'Database', 550.00, 25);
INSERT INTO BOOKS VALUES (BOOK_SEQ.NEXTVAL, 'Database System Concepts', 'Silberschatz', 'Database', 650.00, 15);
INSERT INTO BOOKS VALUES (BOOK_SEQ.NEXTVAL, 'Clean Code', 'Robert C. Martin', 'Programming', 450.00, 20);
INSERT INTO BOOKS VALUES (BOOK_SEQ.NEXTVAL, 'The Pragmatic Programmer', 'Andrew Hunt', 'Programming', 500.00, 10);
INSERT INTO BOOKS VALUES (BOOK_SEQ.NEXTVAL, 'Atomic Habits', 'James Clear', 'Self-Help', 350.00, 30);
INSERT INTO BOOKS VALUES (BOOK_SEQ.NEXTVAL, 'Wings of Fire', 'Dr. A.P.J. Abdul Kalam', 'Biography', 280.00, 18);
INSERT INTO BOOKS VALUES (BOOK_SEQ.NEXTVAL, 'To Kill a Mockingbird', 'Harper Lee', 'Fiction', 300.00, 5); -- Low stock
COMMIT;
-- Register Users via Procedure
BEGIN
    register_user('Rahul Sharma', 'rahul@gmail.com', 'rahul123', '9876543210', 'CUSTOMER');
    register_user('Priya Singh', 'priya@gmail.com', 'priya123', '9123456780', 'CUSTOMER');
    register_user('Admin User', 'admin@bookstore.com', 'admin123', '9998887770', 'ADMIN');
END;
/

-- ------------------------------------------------------------------------
-- STEP 8: TEST RUN (Checking Everything Works)
-- ------------------------------------------------------------------------
PROMPT
PROMPT ==========================================================
PROMPT CURRENT CATALOG IN DATABASE (Via Cursor Procedure):
PROMPT ==========================================================
EXEC display_all_books;

PROMPT
PROMPT ==========================================================
PROMPT SIMULATING SHOPPING & ORDER PLACEMENT
PROMPT ==========================================================
BEGIN
    -- Rahul (User ID 1) adds Book 101 (PL/SQL) and Book 105 (Atomic Habits) to cart
    DBMS_OUTPUT.PUT_LINE('--- 1. Rahul Adding Items to Cart ---');
    add_to_cart(1, 101, 2);   -- 2 copies of PL/SQL book
    add_to_cart(1, 105, 1);   -- 1 copy of Atomic Habits

    -- Place order
    DBMS_OUTPUT.PUT_LINE('--- 2. Placing Order for Rahul ---');
    place_order(1);
END;
/

PROMPT
PROMPT ==========================================================
PROMPT CATALOG STOCK AFTER ORDER (Stock Reduced automatically):
PROMPT ==========================================================
EXEC display_all_books;

PROMPT
PROMPT ==========================================================
PROMPT TESTING TRIGGER: CANCELLING ORDER □#5001 (Stock Restores)
PROMPT ==========================================================
UPDATE ORDERS SET order_status = 'CANCELLED' WHERE order_id = 5001;
COMMIT;

PROMPT
PROMPT ==========================================================
PROMPT CATALOG STOCK AFTER CANCELLATION (Restored via Trigger):
PROMPT ==========================================================
EXEC display_all_books;

PROMPT ==================================================
PROMPT ALL DONE! SCRIPT FINISHED SUCCESSFULLY!
PROMPT ==================================================
```

app.py:

```
#!/usr/bin/env python3
"""
======================================================================
ONLINE BOOKSTORE MANAGEMENT SYSTEM (STUDENT MINI-PROJECT BACKEND)
======================================================================
Simple, clean Python backend (Zero extra libraries needed).
Uses standard Python library (http.server + sqlite3) to run the student UI.
======================================================================
"""

import os
import json
import sqlite3
import mimetypes
from urllib.parse import urlparse, parse_qs
from http.server import HTTPServer, BaseHTTPRequestHandler

PORT = 5000
DB_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "student_bookstore.db")
PUBLIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "public")

def get_db():
    conn = sqlite3.connect(DB_FILE)
    conn.row_factory = sqlite3.Row
    return conn

def init_db():
    conn = get_db()
    c = conn.cursor()

    # 1. Create 5 Simple Student Tables
    c.executescript("""
    CREATE TABLE IF NOT EXISTS USERS (
        user_id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        email TEXT UNIQUE NOT NULL,
        password TEXT NOT NULL,
        phone TEXT,
        role TEXT DEFAULT 'CUSTOMER'
    );

    CREATE TABLE IF NOT EXISTS BOOKS (
        book_id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        author TEXT NOT NULL,
        category TEXT,
        price REAL NOT NULL,
        stock INTEGER NOT NULL DEFAULT 0
    );

    CREATE TABLE IF NOT EXISTS CART (
        cart_id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_id INTEGER REFERENCES USERS(user_id),
        book_id INTEGER REFERENCES BOOKS(book_id),
        quantity INTEGER DEFAULT 1
    );

    CREATE TABLE IF NOT EXISTS ORDERS (
        order_id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_id INTEGER REFERENCES USERS(user_id),
        order_date TEXT DEFAULT CURRENT_TIMESTAMP,
        total_amount REAL NOT NULL,
        order_status TEXT DEFAULT 'PLACED'
    );

    CREATE TABLE IF NOT EXISTS ORDER_ITEMS (
        item_id INTEGER PRIMARY KEY AUTOINCREMENT,
        order_id INTEGER REFERENCES ORDERS(order_id),
        book_id INTEGER REFERENCES BOOKS(book_id),
        quantity INTEGER NOT NULL,
        unit_price REAL NOT NULL
    );
    """)

    # 2. Seed Sample Users
    c.execute("SELECT COUNT(*) FROM USERS")
    if c.fetchone()[0] == 0:
        c.executemany("INSERT INTO USERS (name, email, password, phone, role) VALUES (?, ?, ?, ?, ?)", [
            ("Rahul Sharma", "rahul@gmail.com", "rahul123", "9876543210", "CUSTOMER"),
            ("Priya Singh", "priya@gmail.com", "priya123", "9123456780", "CUSTOMER"),
            ("Admin", "admin@bookstore.com", "admin123", "9998887770", "ADMIN")
        ])

    # 3. Seed Sample Books
    c.execute("SELECT COUNT(*) FROM BOOKS")
    if c.fetchone()[0] == 0:
        c.executemany("INSERT INTO BOOKS (title, author, category, price, stock) VALUES (?, ?, ?, ?, ?)", [
            ("Oracle PL/SQL Programming", "Steven Feuerstein", "Database", 550.00, 25),
            ("Database System Concepts", "Abraham Silberschatz", "Database", 650.00, 15),
            ("Clean Code", "Robert C. Martin", "Programming", 450.00, 20),
            ("The Pragmatic Programmer", "Andrew Hunt", "Programming", 500.00, 10),
            ("Atomic Habits", "James Clear", "Self-Help", 350.00, 30),
            ("Wings of Fire", "Dr. A.P.J. Abdul Kalam", "Biography", 280.00, 18),
            ("To Kill a Mockingbird", "Harper Lee", "Fiction", 300.00, 5)
        ])

    conn.commit()
    conn.close()

class StudentRequestHandler(BaseHTTPRequestHandler):
```

```
    def send_json(self, data, status=200):
        body = json.dumps(data).encode("utf-8")
        self.send_response(status)
        self.send_header("Content-Type", "application/json")
        self.send_header("Access-Control-Allow-Origin", "*")
        self.send_header("Access-Control-Allow-Headers", "*")
        self.send_header("Access-Control-Allow-Methods", "*")
        self.end_headers()
        self.wfile.write(body)

    def do_OPTIONS(self):
        self.send_response(204)
        self.send_header("Access-Control-Allow-Origin", "*")
        self.send_header("Access-Control-Allow-Headers", "*")
        self.send_header("Access-Control-Allow-Methods", "*")
        self.end_headers()

    def do_GET(self):
        parsed = urlparse(self.path)
        path = parsed.path

        # Static files (HTML, CSS, JS)
        if not path.startswith("/api/"):
            if path == "/" or path == "":
                path = "/index.html"
            local_file = os.path.join(PUBLIC_DIR, path.lstrip("/"))
            if os.path.isfile(local_file):
                mime, _ = mimetypes.guess_type(local_file)
                self.send_response(200)
                self.send_header("Content-Type", mime or "text/html")
                with open(local_file, "rb") as f:
                    self.end_headers()
                    self.wfile.write(f.read())
                return
            else:
                self.send_response(404)
                self.end_headers()
                self.wfile.write(b"Not Found")
                return

        conn = get_db()
        c = conn.cursor()

        # 1. Get all books (Catalog)
        if path == "/api/books":
            c.execute("SELECT * FROM BOOKS ORDER BY book_id ASC")
            books = [dict(r) for r in c.fetchall()]
            self.send_json({"status": "SUCCESS", "books": books})

        # 2. Get Cart Items for user (user_id = 1)
        elif path == "/api/cart":
            uid = 1  # Rahul Sharma (Default student user)
            c.execute("""
            SELECT c.cart_id, c.book_id, b.title, b.author, b.price, c.quantity,
                   (c.quantity * b.price) as item_total, b.stock as available_stock
            FROM CART c
            JOIN BOOKS b ON c.book_id = b.book_id
            WHERE c.user_id = ?
            """, (uid,))
            items = [dict(r) for r in c.fetchall()]
            total = sum(i["item_total"] for i in items)
            self.send_json({"status": "SUCCESS", "items": items, "cart_total": total})

        # 3. Get Orders for user
        elif path == "/api/orders":
            uid = 1
            c.execute("""
            SELECT o.*,
                   (SELECT COUNT(*) FROM ORDER_ITEMS WHERE order_id = o.order_id) as total_items
            FROM ORDERS o
            WHERE o.user_id = ?
            ORDER BY o.order_id DESC
            """, (uid,))
            orders = [dict(r) for r in c.fetchall()]
            self.send_json({"status": "SUCCESS", "orders": orders})

        # 4. Get Order Details
        elif path.startswith("/api/orders/"):
            oid = int(path.split("/")[-1])
            c.execute("""
            SELECT oi.*, b.title, b.author
            FROM ORDER_ITEMS oi
            JOIN BOOKS b ON oi.book_id = b.book_id
            WHERE oi.order_id = ?
            """, (oid,))
            items = [dict(r) for r in c.fetchall()]
            self.send_json({"status": "SUCCESS", "items": items})

        conn.close()

    def do_POST(self):
        path = urlparse(self.path).path
        content_length = int(self.headers.get("Content-Length", 0))
        payload = json.loads(self.rfile.read(content_length).decode("utf-8")) if content_length > 0 else {}
        conn = get_db()
        c = conn.cursor()

        # 1. Add to Cart (Procedure: add_to_cart)
        if path == "/api/cart":
```

```
            uid = 1
            bid = payload.get("book_id")
            qty = payload.get("quantity", 1)

            # Check stock
            c.execute("SELECT stock, title FROM BOOKS WHERE book_id = ?", (bid,))
            b = c.fetchone()
            if not b or b["stock"] < qty:
                self.send_json({"status": "ERROR", "message": f"Out of stock! Only {b['stock'] if b else 0} left."})
                conn.close()
                return

            c.execute("SELECT quantity FROM CART WHERE user_id = ? AND book_id = ?", (uid, bid))
            existing = c.fetchone()
            if existing:
                c.execute("UPDATE CART SET quantity = quantity + ? WHERE user_id = ? AND book_id = ?", (qty, uid, bid))
            else:
                c.execute("INSERT INTO CART (user_id, book_id, quantity) VALUES (?, ?, ?)", (uid, bid, qty))

            conn.commit()
            self.send_json({
                "status": "SUCCESS",
                "message": f'Added "{b["title"]}" to cart.',
                "plsql_log": f"EXEC add_to_cart(p_user_id => 1, p_book_id => {bid}, p_qty => {qty});"
            })

        # 2. Place Order (Procedure: place_order)
        elif path == "/api/orders":
            uid = 1
            c.execute("""
            SELECT c.book_id, c.quantity, b.price, b.stock, b.title
            FROM CART c
            JOIN BOOKS b ON c.book_id = b.book_id
            WHERE c.user_id = ?
            """, (uid,))
            cart_items = c.fetchall()

            if not cart_items:
                self.send_json({"status": "ERROR", "message": "Cart is empty."})
                conn.close()
                return

            # Check stock for all
            for it in cart_items:
                if it["stock"] < it["quantity"]:
                    self.send_json({"status": "ERROR", "message": f'Stock insufficient for "{it["title"]}"'})
                    conn.close()
                    return

            total = sum(it["price"] * it["quantity"] for it in cart_items)

            # Insert into ORDERS
            c.execute("INSERT INTO ORDERS (user_id, total_amount, order_status) VALUES (?, ?, 'PLACED')", (uid, total))
            oid = c.lastrowid

            # Insert ORDER_ITEMS & update stock
            for it in cart_items:
                c.execute("INSERT INTO ORDER_ITEMS (order_id, book_id, quantity, unit_price) VALUES (?, ?, ?, ?)",
                          (oid, it["book_id"], it["quantity"], it["price"]))
                c.execute("UPDATE BOOKS SET stock = stock - ? WHERE book_id = ?", (it["quantity"], it["book_id"]))

            # Clear cart
            c.execute("DELETE FROM CART WHERE user_id = ?", (uid,))
            conn.commit()

            self.send_json({
                "status": "SUCCESS",
                "message": f"Order #{oid} placed successfully for Rs. {total:.2f}!",
                "plsql_log": f"EXEC place_order(p_user_id => 1); -- Order #{oid} created, stock deducted, cart cleared."
            })

        # 3. Cancel Order (Trigger: trg_restore_stock_on_cancel)
        elif path.endswith("/cancel"):
            oid = int(path.split("/")[-2])
            c.execute("SELECT order_status FROM ORDERS WHERE order_id = ?", (oid,))
            ord_row = c.fetchone()

            if not ord_row or ord_row["order_status"] == "CANCELLED":
                self.send_json({"status": "ERROR", "message": "Order already cancelled or not found."})
                conn.close()
                return

            # Trigger Logic: restore stock
            c.execute("SELECT book_id, quantity FROM ORDER_ITEMS WHERE order_id = ?", (oid,))
            items = c.fetchall()
            for it in items:
                c.execute("UPDATE BOOKS SET stock = stock + ? WHERE book_id = ?", (it["quantity"], it["book_id"]))

            c.execute("UPDATE ORDERS SET order_status = 'CANCELLED' WHERE order_id = ?", (oid,))
            conn.commit()

            self.send_json({
                "status": "SUCCESS",
                "message": f"Order #{oid} cancelled. Inventory automatically restored!",
                "plsql_log": f"TRIGGER FIRED: trg_restore_stock_on_cancel on Order #{oid} -> Stock added back to BOOKS table."
            })
```

```
296
297         # 4. Add Book (Admin)
298         elif path == "/api/books":
299             title = payload.get("title")
300             author = payload.get("author")
301             category = payload.get("category", "General")
302             price = float(payload.get("price", 100))
303             stock = int(payload.get("stock", 10))
304
305             c.execute("INSERT INTO BOOKS (title, author, category, price, stock) VALUES (?, ?, ?, ?, ?)",
306                       (title, author, category, price, stock))
307             bid = c.lastrowid
308             conn.commit()
309             self.send_json({
310                 "status": "SUCCESS",
311                 "message": f'New book "{title}" added with ID #{bid}!',
312                 "plsql_log": f"INSERT INTO BOOKS VALUES ({bid}, '{title}', '{author}', '{category}', {price}, {stock});"
313             })
314
315         conn.close()
316
317     def do_DELETE(self):
318         path = urlparse(self.path).path
319         conn = get_db()
320         c = conn.cursor()
321
322         if path.startswith("/api/cart/"):
323             cart_id = int(path.split("/")[-1])
324             c.execute("DELETE FROM CART WHERE cart_id = ?", (cart_id,))
325             conn.commit()
326             self.send_json({"status": "SUCCESS", "message": "Item removed from cart."})
327
328         conn.close()
329
330 def main():
331     init_db()
332     os.makedirs(PUBLIC_DIR, exist_ok=True)
333     server_address = ("127.0.0.1", PORT)
334     httpd = HTTPServer(server_address, StudentRequestHandler)
335     print("=" * 65)
336     print("  ONLINE BOOKSTORE MANAGEMENT SYSTEM (PL/SQL MINI PROJECT)")
337     print(f"  Local Web App URL: http://localhost:{PORT}")
338     print("=" * 65)
339     print("Press Ctrl + C in VS Code terminal to stop the server.\n")
340     try:
341         httpd.serve_forever()
342     except KeyboardInterrupt:
343         print("\nServer stopped.")
344         httpd.server_close()
345
346 if __name__ == "__main__":
347     main()
348
```

Result:

	book_id	title	author	category	price	stock
	Filte	Filter...	Filter...	Filte	Filte	Filte
1	1	Oracle PL/SQL Programming	Steven Feuerstein	Database	550	24
2	2	Database System Concepts	Abraham Silberschatz	Database	650	14
3	3	Clean Code	Robert C. Martin	Programming	450	20
4	4	The Pragmatic Programmer	Andrew Hunt	Programming	500	10
5	5	Atomic Habits	James Clear	Self-Help	350	30
6	6	Wings of Fire	Dr. A.P.J. Abdul Kalam	Biography	280	18
7	7	To Kill a Mockingbird	Harper Lee	Fiction	300	5
8	8	The Mountain is you	Brianna Wiest	Self-Help	350	15
9						

Figure 1: After inserting Data

	cart_id	user_id	book_id	quantity
	Filte	Filte	Filte	Filte
1	5	1	8	2
2	6	1	1	1
3				

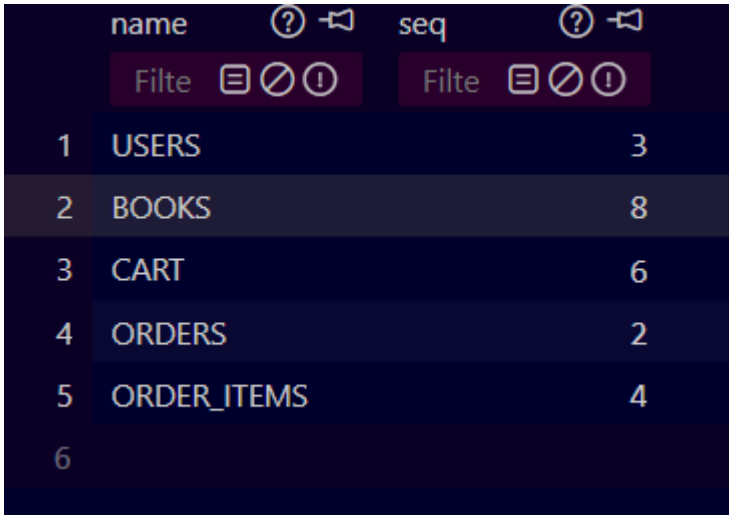
Figure 2: After Inserting Data

	item_id	order_...	book_id	quantity	unit_price
	Filte	Filte	Filte	Filte	Filte
1	1	1	1	1	550
2	2	1	2	1	650
3	3	2	1	1	550
4	4	2	2	1	650
5					

Figure 3: After Inserting Data

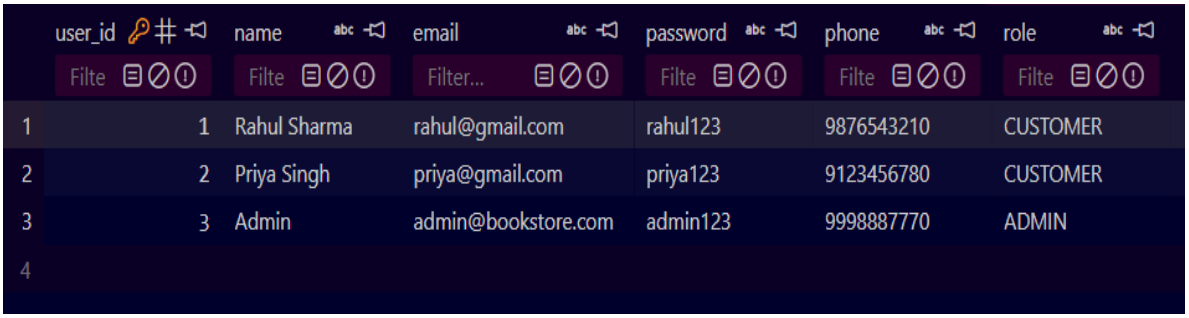
	order_...	user_id	order_date	total_am...	order_sta...
	Filte	Filte	Filter...	Filte	Filte
1	1	1	2026-10-27 14:12:55	1200	CANCELLED
2	2	1	2026-10-27 18:16:38	1200	PLACED
3					

Figure 4: After Inserting Data



	name	seq
1	USERS	3
2	BOOKS	8
3	CART	6
4	ORDERS	2
5	ORDER_ITEMS	4
6		

Figure 5: After Inserting Data



	user_id	name	email	password	phone	role
1	1	Rahul Sharma	rahul@gmail.com	rahul123	9876543210	CUSTOMER
2	2	Priya Singh	priya@gmail.com	priya123	9123456780	CUSTOMER
3	3	Admin	admin@bookstore.com	admin123	9998887770	ADMIN
4						

Figure 6: After Inserting Data

6. Conclusion

The **Medi Chain** project successfully demonstrates the application of **Advanced Database Management System (ADBMS)** concepts in developing a secure, efficient, and scalable **hospital management system**. By using a **bottom-up design approach**, the project effectively integrates various entities such as patients, doctors, appointments, and billing into a unified structure that ensures smooth data management and retrieval. The implementation of **normalization** eliminates redundancy and maintains data consistency, while **fragmentation** and **distributed view management** enhance system performance and scalability across multiple hospital branches.

Through the use of **relational algebra operations**, Medi Chain efficiently handles data queries and relationships among different entities. The inclusion of a **transaction manager** ensures reliability and consistency by maintaining **ACID properties** during simultaneous transactions. Overall, Medi Chain not only simplifies hospital operations but also improves data security and accessibility in a distributed environment. The project demonstrates how ADBMS techniques can be effectively applied to real-world healthcare management, providing a strong foundation for future enhancements such as cloud integration and AI-based medical data analytics.

References

1. GitHub– https://github.com/rhydham001/MediChain
2. MySQL Reference Manual – https://dev.mysql.com/doc/
3. Fundamentals of Database Systems –https://www.w3schools.com/sql
4. Advanced Database Management System (ADBMS) Concepts– https://www.tutorialspoint.com/
5. Database ER Diagram Design Tool– https://app.diagrams.net/
6. Transaction Management in DBMS–

   https://www.geeksforgeeks.org